Verified admission design

Date: 2026-10-22. **Owner-approved architectural amendment, 2026-10-22; implementation planning only.** No code, remote settings, identity grants, credentials, or activation are included. This proposal addresses the native Ready identity gap in the runtime capability record.

Contents

Portable approval record
Recommendation and exact amendment
Options and user experience
Proposed handshake
What the proof establishes
Failure handling and permissions
Acceptance tests and remaining blockers
Owner decision

Portable approval record

Repository copy of the owner-approved 2026-10-22 amendment and implementation plan. Approval-source SHA256: design `b12db59636fe472fc494aef41a66396dc7b1bdfcbe32cd32b5a9ffff391be873`; original approved plan `892e1300716085e34341da35c5ea9a7a477a0945d7eb16113f2e8f88fc5bbf16`; approval-metadata plan revision `e5f98233dbb9fb92b6a2705acbd1fd2cbeaded994dacb23161afc92a1c9fa568`.

This portable revision changes approval metadata and links only. Supporting links point to repository-maintained capability records rather than local planning evidence. Requirements remain the approved amendment; historical approval and prerequisite descriptions are not claims of live activation or hosted acceptance. Offline implementation is staged on a Draft branch, pending exact-source review and delivery. Final exports follow the project PDF convention.

Recommendation and exact amendment

Recommend a **Ready with verification** command that deliberately requests admission of an exact base/head pair. A trusted controller observes Draft, records that pair and native history, marks the PR Ready, then verifies current Ready status and a newly observed native Ready event before persisting admission. Workers still require that same latest native Ready event and exact pair.

This is a small but material change to the proof requirement. The admission record proves **which exact pair the authorized request selected and the controller verified while observing native Ready**. It does **not** prove that pair was GitHub's base/head at the instantaneous native Ready event. Before/after reads cannot establish uninterrupted equality between them. The owner must approve this distinction; implementation must not describe it as closing the original historical-payload join.

Proposed policy replacement:

CI admission requires an authorized, candidate-specific readiness request, a verified trusted-controller handshake, current native Ready status, the handshake's latest native Ready event witness, and the same exact base/head pair. The candidate is bound at verified admission, not reconstructed as an event-time snapshot. Ambiguity or any detected intervening candidate/readiness change denies admission. All worker starts and consequential completions revalidate these facts. Unknown outcomes remain pending; no automatic restoration to Ready is permitted.

The native Ready ID remains part of the candidate identity. Add a single-use request/operation ID and protocol version; do not relabel a workflow run ID as a native event ID. Preserve the initiating human identity separately from the automation actor that changes the native state. No merge authority changes: `glyad` remains the sole merger.

Options and user experience

Option	User action and consequence	Assessment
A - Verified handshake (recommended)	While Draft, run **Ready with verification** for the PR and the exact locally validated base/head. The controller either reports "Ready and admitted for these commits" or "Not admitted; reconciliation required."	Gives a concrete automation path using supported APIs, subject to the explicit admission-time amendment and remaining backend/provenance gates.
B - Keep native Ready button and original proof contract	Use GitHub's existing button. If the event-time pair/latest-event join cannot be proven, CI stays blocked. A human returns the PR to Draft and resolves the missing evidence; a manual "looks correct" flag cannot supply the absent proof.	Smallest policy change: none. Does not guarantee eventual automatic admission, even after another click.

Initial command UX: use the Actions page or an authenticated command wrapper to select the PR and submit the expected base/head produced by completed local validation. The wrapper may fill those values; it must not silently choose a newer candidate after dispatch. The workflow runs only an approved default-branch definition. No second confirmation is required within a valid deliberate request. GitHub supports manual workflow runs through UI/CLI/API,

but also allows choosing another ref; therefore “the file exists on default” alone is not a definition-trust check. Manual workflow behavior, dispatch event semantics.

Under A, using the native Ready button without a valid handshake still makes the PR natively Ready, but it does not admit CI. Show a clear blocked/reconciliation status. The recommended fallback leaves it blocked; the human returns it to Draft before issuing the verified command. Do not introduce an automatic Draft→Ready loop or silently adopt a manual Ready event into a different request.

Proposed handshake

These steps specify application checks, not a GitHub transaction. The shared durable backend and exact-run provenance remain prerequisites from the existing gap map.

1. **Authenticate the deliberate request.** Bind repository/PR IDs, expected base ref/SHA, head SHA, request nonce, initiating actor, trusted workflow ID/definition SHA and original run ID. Restrict request actors to the configured readiness-authorized set. A rerun uses the same request identity and may reconcile it; it cannot create a new readiness attempt or paid reservation. Treat dispatch inputs as untrusted data.
2. **Acquire the shared per-PR serialization domain.** Confirm no unresolved admission/invalidation obligation; obtain required funding-state coordination only when reserving expensive work. Reconcile previous operations first. A runner-local directory or Actions queue alone is insufficient.
3. **Observe and persist the precondition.** Read the current open Draft PR and native timeline. Require exact equality with the expected pair and base ref. Record the observed native Ready/Draft/close/reopen boundary, completeness information and evidence digest in a durable intent before making a mutation. Paginated reads must detect instability; a bounded read failure denies progress.
4. **Make one Ready request.** Immediately revalidate the precondition, then call `markPullRequestReadyForReview` with the PR ID and operation ID as `clientMutationId`. Record the response or an unknown outcome. Do not assume `clientMutationId` provides idempotency or appears in the native event. GitHub returns the PR and echoed client value; it does not return a Ready event ID or accept expected base/head conditions. Ready mutation contract.
5. **Verify an observed transition window.** Read the native history and PR again. Require open Ready, the exact expected pair, exactly one newly observed Ready event since the recorded boundary, the expected automation actor, and no observed intervening Draft/close/reopen/candidate drift. Recheck that this is still the latest Ready event. Missing pages, extra transitions, different actor, mismatch, missing event or unsettled response deny admission. A single new event is a witness to an observed transition window; it is not by itself proof that this HTTP request uniquely caused it.
6. **Persist the proof before dispatch.** Publish and read back authenticated evidence binding the request, observations, mutation result, native Ready witness, exact pair and run/attempt/definition. Commit the durable admission/claim state only after verification succeeds. A crash between native Ready and durable acceptance leaves an unadmitted pending operation. Recovery may reconcile that original operation, never issue another Ready mutation blindly.
7. **Start CI through the existing candidate gate.** Revalidate again immediately before worker dispatch and at worker start/completion. Use explicit trusted dispatch or a connected continuation; do not depend on a bot-generated Ready event triggering another workflow. GitHub suppresses most events generated by `GITHUB_TOKEN`; workflow/repository dispatch are documented exceptions. CI success still precedes independent review, whose funding/key/limits remain separately inactive. Token-trigger behavior.

What the proof establishes

The manifest can truthfully attest: the human requested pair `B/H`; the trusted controller observed Draft at `B/H`, requested Ready, then observed Ready at `B/H` with native event `R`; the admission refers only to `(repository, PR, base ref, B, H, R, operation, protocol version)`. It can retain original run and rerun-attempt attribution without granting duplicate execution.

It cannot establish an atomic snapshot across API calls, prevent a human or another API actor changing the PR, or prove that no unobserved change occurred between reads. In particular, `B/H → B2/H2 → B/H` between observations can defeat a claim of uninterrupted historical equality. The recommendation accepts this **admission-time** contract and the already approved revalidation/quarantine race boundary; it does not certify an event-time pair.

The Ready timeline object exposes actor, ID and time, but not event-time SHAs or the caller's mutation ID. Actor matching is also limited: several workflows may use the same `github-actions[bot]` identity. Inventory all code/identities able to act as the selected controller, route their readiness writes through the same serialization protocol, and reject unrecognized provenance. A check in a workflow controlled by another branch is not an access-control boundary. If exclusive actor coverage cannot be established, do not claim unique causal attribution; block activation if that attribution is still required. Ready timeline fields.

Stable repeated reads reduce ambiguity but do not prove timeline linearizability or replace platform guarantees. If the owner requires exact original event-time base/head or absolute exclusion of all competing native mutations, **neither A nor a manual override proves that requirement**. Retain the original fail-closed gate and investigate a different supported trust boundary.

Failure handling and permissions

- A stale worker result, incomplete proof, mutation race or mismatch invalidates only the recorded attempt and blocks further admission. It never marks another attempt Ready or restores Ready automatically.
- Native Ready without accepted proof remains unadmitted. Default recovery is human reconciliation and a deliberate return to Draft. Any existing automatic Draft safety action retains its pre/post revalidation and quarantine boundary; this proposal does not strengthen it into compare-and-swap.
- Persist uncertain mutation/dispatch outcomes before releasing application ownership. No timeout, rerun or queue replacement clears an uncertainty or refunds a reservation.
- A successful Ready mutation is an externally visible state change and may notify reviewers. Approval of A would authorize that behavior only for an authenticated, deliberate readiness command after activation; it would not authorize arbitrary agent readiness or merging.

Expected permissions are PR metadata/readiness writes, ledger Issues writes, provenance Actions reads, and the already contemplated worker-dispatch capability. The implementation plan must verify the exact effective GitHub permissions. It may reuse a supported repository token; no PAT, App, signing key or external service is selected here. Attestation permissions, storage Contents writes or an isolated App identity remain separate decisions if the chosen production protocol needs them. A command running on `workflow_dispatch` does not remove the existing event-controller's policy/activation requirements.

Acceptance tests and remaining blockers

Before production acceptance, test at least:

Case	Required result
Pair changed while command waits; base-only drift	Reject without retargeting the human request.
Two command runs or a rerun of the first	One durable operation/claim; no duplicate Ready mutation, CI or reservation.
Native Ready before step 3; another actor marks Ready between steps	Reject or quarantine; do not adopt an unexplained transition.
Ready→Draft→Ready, closure/reopen, multiple new Ready events or missing pages	Reject; no timestamp/arrival-order shortcut.
Shared-bot writer outside serialization; untrusted dispatch ref	Reject provenance/activation coverage; actor name is insufficient.
Crash or lost response after Ready; proof upload/write fails	Persist/recover pending state; no CI before authenticated acceptance.
Head/base changes immediately after acceptance or before result posting	Worker gate rejects stale candidate; no automatic Ready restoration.
ABA candidate changes between reads	Never claim event-time equality. If this must be detected universally, mark A infeasible under that stronger requirement.
Artifact deletion/expiry, altered journal body, later-tail deletion	Block until authenticated recovery; no fresh budget inferred.
Bot Ready produces no downstream event	Explicit gated continuation works without adding credentials merely to trigger events.

Approval of this proposal is not proof of those tests or live feasibility. Shared cross-runner storage, writer coverage, trusted definition/run/byte provenance, live API permission checks and the residual-race handling remain activation blockers. Preparing the implementation plan and offline falsification tests is the next stage only after the owner approves this written amendment. The brainstorming skill requires: “written-spec approval only permits invoking writing-plans.” No implementation is requested or authorized by this document.

Owner decision

Proposed single question for the parent chat to present:

Approve “Ready with verification,” which verifies the exact selected commits at admission, or keep the native Ready button and block automation whenever historical proof is missing?

- **A - Approve verified admission (recommended):** approve this narrow written amendment and proceed to its implementation plan; activation remains gated.
- **B - Keep original proof requirement:** preserve the current policy and accept blocked automation/manual reconciliation where proof is unavailable.

Self-review: checked scope, exact identity semantics, actor coverage, ABA/mutation races, bot-event suppression, durable uncertainty, no automatic Ready restoration, sole human merge authority and separate activation gates. This draft makes no unsupported CAS, event-causality, native-signature or completion claim.